

System 프로젝트 — 전체 Flow 상세 설명

- 작성일: 2026-04-15
- 대상: [apps/system/](#) (Link 노드, RAK4631, S/N 001050234191)

1. 프로젝트 구성

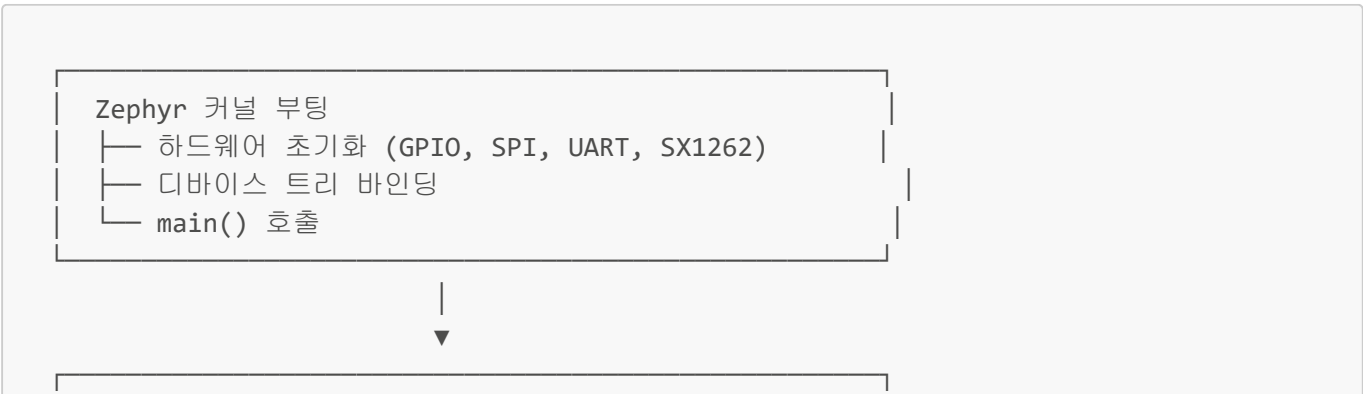
1.1 파일 구조

```
apps/  
├─ lora_byte_proto.h      ← 16B 프레임 직렬화 (공유 헤더)  
└─ system/  
    ├─ CMakeLists.txt  
    ├─ prj.conf  
    ├─ boards/rak4631.overlay ← 디버그 콘솔 → uart1  
    └─ src/  
        ├─ main.c          ← 엔트리포인트  
        ├─ lora_module.c    ← LoRa 송수신 + ACK 상태머신  
        ├─ lora_module.h    ← LoRa 모듈 API  
        ├─ telemetry_task.c ← 주기 업링크 (5초)  
        └─ telemetry_task.h ← 텔레메트리 API
```

1.2 빌드 설정

항목	값
Board	rak4631 (nRF52840 + SX1262)
콘솔 UART	uart1 (P0.16 TX / P0.15 RX, 115200bps)
빌드 출력	/tmp/build_system/
Main 스택	4096 bytes
System Workqueue 스택	2048 bytes

2. 부팅 시퀀스



```
main.c:main()
① LOG_INF("LoRa byte protocol v2 - ...")
② lora_module_init() → LoRa 하드웨어 + 스레드
③ lora_module_set_state(MODULE_STATE_ACTIVE)
④ return 0 (Zephyr idle 스레드로 전환)
```

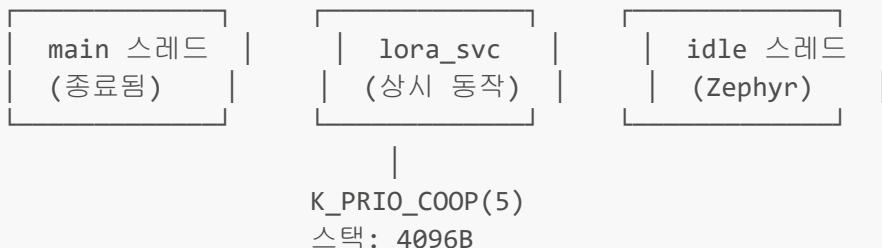
2.1 lora_module_init() 상세 (lora_module.c:306~331)

```
lora_module_init()
├── DEVICE_DT_GET(lora0) → SX1262 디바이스 가져오기
├── device_is_ready() 검증
├── k_mutex_init(&lora_core_mutex)
├── k_thread_create("lora_svc") → lora_service_thread 생성
│   ├── 우선순위: K_PRIO_COOP(5), 스택: 4096B
├── enter_rx_continuous() → RX 모드 진입
│   └── ※ 여기서 -16 에러 발생 가능 (스레드와 경합, 동작 무영향)
└── LOG_INF("LoRa v2 16B frame, device node_id=0x001F gateway=0x0001")
```

참고: `main()`에서 `telemetry_task_init()`는 현재 호출하지 않는다. 현재 버전은 `CREATE(sensor)` 하향 수신 전용이다.

3. 스레드 구조

부팅 후 시스템에는 다음 스레드들이 동작한다:



4. lora_service_thread — 서비스 루프 상세

이것이 시스템의 **핵심 루프**이다 (lora_module.c:241~303).

```
lora_service_thread()
├── enter_rx_continuous() ← 초기 RX 모드
└── for (;;) ← 무한 루프
    ├── ① TX 큐 확인 (K_NO_WAIT)
```

```

    k_msgq_get(&lora_tx_queue, &tx_item, K_NO_WAIT)
    └─ 있으면 → lora_do_tx()

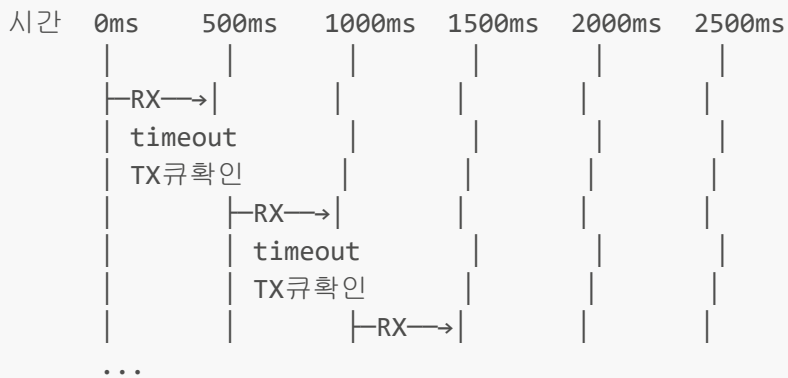
    ─ ② RX 타임아웃 계산
      └─ ACK 대기 중 → 남은 ACK 시간 (최대 2000ms)
      └─ ACK 대기 아님 → 500ms

    ─ ③ lora_rcv(rx_timeout) ← 블로킹 수신
      └─ n > 0 → 패킷 수신 → lora_handle_rx_buf()
      └─ n == -EAGAIN → 정상 타임아웃 (무시)
      └─ n < 0 (기타) → 에러 로그 + 100ms 대기 + RX 재진입

    └─ (루프 반복)

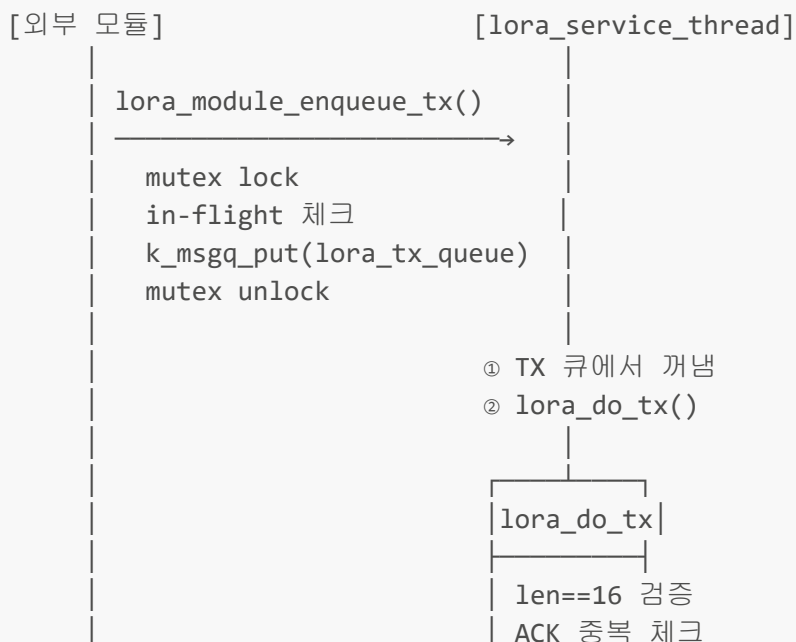
```

4.1 시간 흐름 예시 (정상 동작)



5. TX 경로 — 데이터 송신 흐름

5.1 외부 → TX 큐 → 송신



```

lora_radio_send()
├─ apply_modem_config(tx=true)
├─ lora_send(data, 16)
└─ enter_rx_continuous()

need_ack?
├─ YES → inflight 저장
│   │   waiting_ack = true
│   │   ack_start_ms 기록
│   │   → WAIT_ACK 상태
└─ NO → IDLE 상태

```

5.2 단일 in-flight 규칙

```

ACK 대기 중 (lora_tx_waiting_ack == true)
├─ 새 TX 큐잉 시도 → 거절 (return false)
│   │   "TX dropped: already waiting for ACK"

```

- 한 번에 하나의 DATA만 ACK 대기 가능
- ACK 수신 또는 재전송 실패 후에야 다음 DATA 송신 가능

6. RX 경로 — 데이터 수신 흐름

6.1 lora_handle_rx_buf() 처리 순서 (lora_module.c:115~158)

```

lora_recv() → n > 0 → lora_handle_rx_buf(buf, len)
├─ len < 16 → 무시 (return)
├─ ① node_id 필터 (프로토콜 §1)
│   │   dest_node_id = buf[0:1] (Big-Endian)
│   │   dest != LORA_DEVICE_NODE_ID(0x001F) → 무시 + rejected 카운트
├─ ② ACK 매칭 (ACK power)
│   │   lora_proto_match_ack_power() → true
│   │   └─ lora_handle_ack_received()
│   │       │   waiting_ack = false
│   │       │   retry_count = 0
│   │       │   state = IDLE
│   │       └─ ack_callback(true) → LOG_INF("ACK: OK")
├─ ③ CREATE(sensor) 하향 수신
│   │   lora_proto_match_create_sensor_for_device() → true
│   │   └─ device_index 추출 (buf[4])
│   │       │   LOG_INF("RX CREATE sensor pkt=%u device_index=%u")
│   │       │   ACK(sensor) 인코딩
│   │       └─ lora_radio_send(ack) → 즉시 응답

```

④ 기타 프레임

lora_rx_other_count++ (카운트만)

7. ACK 타임아웃 + 재전송 흐름

DATA 송신 완료 (need_ack=true)

ack_start_ms = k_uptime_get_32()

◆ 매 RX 루프마다 경과 시간 체크:

elapsed < 2000ms → RX 계속 (남은 시간만큼 블로킹)

└ 이 사이에 ACK 수신 → lora_handle_ack_received() → 종료

elapsed >= 2000ms → ACK 타임아웃!

└ retry_count < 3 → 재전송

└ | retry_count++

└ | LOG_WRN("ACK timeout, retry N/3")

└ | lora_radio_send(inflight_data)

└ | ack_start_ms 갱신 → 다시 대기

└ retry_count >= 3 → 최종 실패

└ | LOG_ERR("ACK failed after 3 retries")

└ | lora_handle_ack_failure()

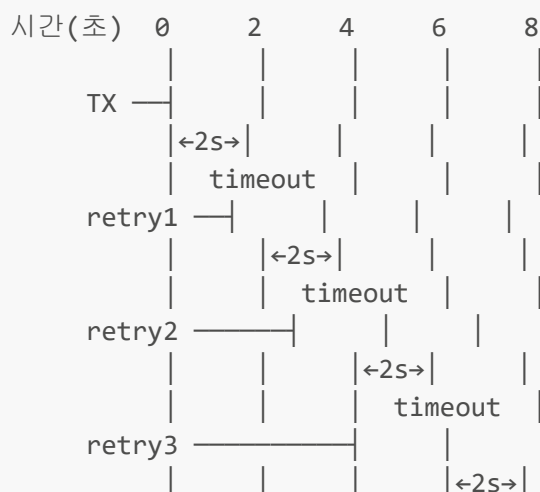
└ | waiting_ack = false

└ | retry_count = 0

└ | state = IDLE

└ | ack_callback(false) → LOG_INF("ACK: FAIL")

7.1 타이밍 다이어그램



timeout
FAIL

8. 라디오 모드 전환

SX1262는 TX와 RX를 동시에 할 수 없다. `lora_radio_send()`가 모드를 관리한다.

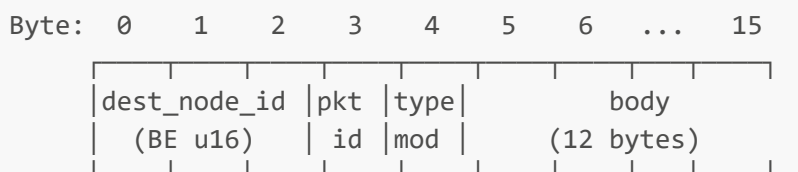
```
lora_radio_send(data, len)
├─ apply_modem_config(tx=true)  ← TX 모드 전환
├─ lora_send(data, len)         ← 실제 RF 송신
└─ enter_rx_continuous()       ← 즉시 RX 모드 복귀
```

RF 설정 (`apply_modem_config`):

```
frequency: 922 MHz
bandwidth: BW_125_KHZ
datarate: SF_7
coding_rate: CR_4_5
preamble_len: 8
tx_power: 14 dBm
iq_inverted: false
public_network: false
```

9. 프로토콜 프레임 구조

`lora_byte_proto.h`에서 정의하는 16바이트 고정 프레임:



9.1 현재 구현된 메시지 (4종)

메시지	type_module	방향	body
DATA battery_mv	0x63	Link→Tower	[4-5] battery_mv (BE u16)
ACK power	0x03	Tower→Link	전부 0
CREATE sensor	0x12	Tower→Link	[4] device_index
ACK sensor	0x02	Link→Tower	전부 0

9.2 node_id 매핑

역할	node_id	매크로
Link (디바이스)	0x001F	LORA_DEVICE_NODE_ID
Tower (게이트웨이)	0x0001	LORA_GATEWAY_NODE_ID

10. 상태 관리

10.1 ModuleRunState (모듈 활성/비활성)

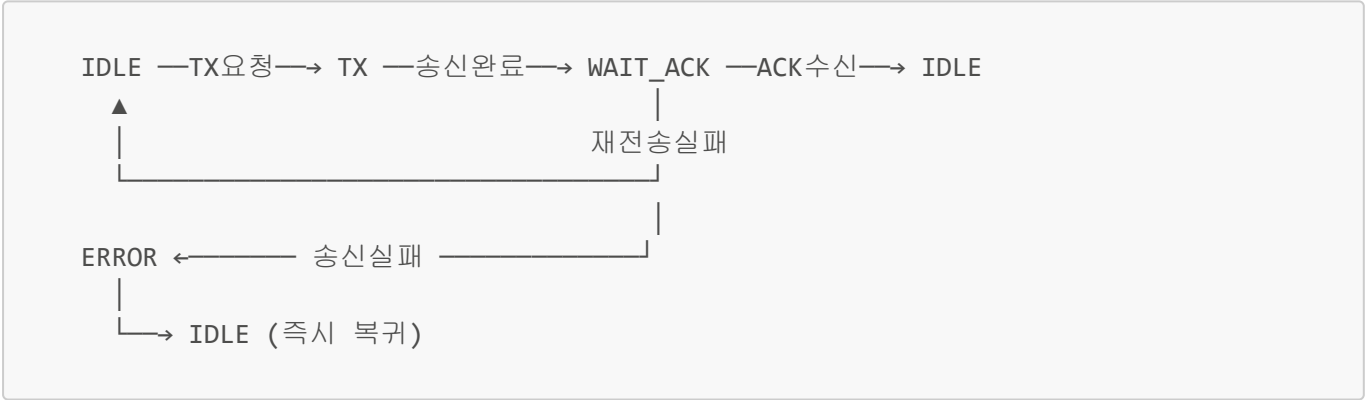
```
lora_module_set_state()

- INACTIVE → 큐 퍼지, ACK 초기화, IDLE 전환
- ACTIVE → IDLE 전환, RX 모드 진입

```

- INACTIVE 상태에서는 TX 큐에서 꺼내도 송신하지 않음
- RX 데이터가 와도 처리하지 않음

10.2 LoRaState (내부 상태머신)



11. 동기화 메커니즘

자원	보호 방식	용도
lora_core_mutex	k_mutex	lora_run_state, lora_state, ACK 변수, TX 실행 보호
lora_tx_queue	k_msgq (depth=8)	외부 모듈 → lora_svc 스레드 간 메시지 전달

- TX 큐: **K_NO_WAIT**로 넣기 (enqueue), **K_NO_WAIT**로 꺼내기 (dequeue)
- 뮤텍스: **K_FOREVER**로 획득 (블로킹)

12. 현재 제약 사항

항목	현재 상태	비고
구현 메시지	4/19종	Phase 1에서 19종 확장 예정
RX 구조	lora_rcv 500ms 폴링	목표: 큐 전용 대기 + RX 인터럽트

항목	현재 상태	비고
node_id	매크로 고정 (0x001F)	Phase 2에서 Flash/NVS 읽기 예정
텔레메트리	main에서 미호출	CREATE 수신 전용 모드
RX -16 에러	부팅 시 발생	동작 무영향, 원인 미조사

작성: Claude Code